

Exercise 2: E-commerce Platform Search Function

1. Understanding Asymptotic Notation

What is Big O Notation?

Big O Notation measures the efficiency of an algorithm as the input size increases.

It helps us:

- Compare algorithms.
- Predict performance.
- Choose the best solution.

Common Complexities

$O(1)$ Constant Time

$O(\log n)$ Logarithmic Time

$O(n)$ Linear Time

$O(n \log n)$ Efficient Sorting

$O(n^2)$ Nested Loops

Search Cases

Best Case

Item found at first position.

Time Complexity = $O(1)$

Average Case

Item found somewhere in the middle.

Time Complexity = $O(n)$

Worst Case

Item found at last position or not found.

Time Complexity = $O(n)$

2. Product Class

```
public class Product {
    private int productId;
    private String productName;
    private String category;
    public Product(int productId,
                   String productName,
                   String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
    public int getProductId() {
        return productId;
    }
    public String getProductName() {
        return productName;
    }

    }
    public String getCategory() {
        return category;
    }
    }
    @Override
    public String toString() {
        return "ID : " + productId +
            " Name : " + productName +
            " Category : " + category;
    }
}
```

3. Linear Search Implementation

Search Class

```
public class LinearSearch {
    public static Product search(
        Product[] products,
        String targetName) {
        for(Product product : products) {
            if(product.getProductName()
                .equalsIgnoreCase(targetName)) {
                return product;
            }
        }
        return null;
    }
}
```

4. Binary Search Implementation

Product Array Must Be Sorted

```
import java.util.Arrays;
import java.util.Comparator;
public class BinarySearch {
    public static Product search(
        Product[] products,
        String targetName) {
        int left = 0;
        int right = products.length - 1;
        while(left <= right) {
            int mid =
                (left + right) / 2;
            int result =
                products[mid]
                .getProductName()
                .compareToIgnoreCase(targetName);
            if(result == 0) {
                return products[mid];
            }
            else if(result < 0) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return null;
    }
}
```

5. Main Program

```
import java.util.Arrays;
import java.util.Comparator;
public class SearchTest {
    public static void main(String[] args) {
        Product[] products = {
            new Product(
                101,
                "Laptop",
                "Electronics"),
            new Product(
                102,
                "Mobile",
                "Electronics"),
            new Product(
                103,
```

```

        "Shoes",
        "Fashion"),
    new Product(
        104,
        "Watch",
        "Accessories")
};
System.out.println(
    "LINEAR SEARCH");
Product result1 =
    LinearSearch.search(
        products,
        "Shoes");
if(result1 != null)
    System.out.println(result1);
Arrays.sort(
    products,
    Comparator.comparing(
        Product::getProductName));
System.out.println(
    "\nBINARY SEARCH");
Product result2 =
    BinarySearch.search(
        products,
        "Watch");
if(result2 != null)
    System.out.println(result2);
}
}

```

Output

LINEAR SEARCH

ID : 103

Name : Shoes

Category : Fashion

BINARY SEARCH

ID : 104

Name : Watch

Category : Accessories

6. Time Complexity Analysis

Linear Search

Case	Complexity
------	------------

Best	$O(1)$
------	--------

Average	$O(n)$
---------	--------

Worst	$O(n)$
-------	--------

Binary Search

Case	Complexity
------	------------

Best	$O(1)$
------	--------

Average	$O(\log n)$
---------	-------------

Worst	$O(\log n)$
-------	-------------

7. Comparison

Feature	Linear Search	Binary Search
Data Requirement	Unsorted	Sorted
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Large Data Performance	Slow	Fast

8. Which Algorithm is Better?

Linear Search

Use when:

- Data is small.
- Data is unsorted.

Binary Search

Use when:

- Data is large.
- Data is sorted.
- High search performance is required.

For an **E-commerce Platform**, **Binary Search is more suitable** because product catalogs can contain thousands or millions of products and searching must be fast.

Exercise 7: Financial Forecasting (Recursion)

1. Understanding Recursive Algorithms

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a smaller version of the same problem.

A recursive function contains:

1. **Base Case** – stops recursion.
2. **Recursive Case** – function calls itself.

Example

```
public static int factorial(int n){  
  
    if(n == 0)  
        return 1;  
  
    return n * factorial(n - 1);  
}
```

Working

factorial(5)

$5 \times \text{factorial}(4)$
 $4 \times \text{factorial}(3)$
 $3 \times \text{factorial}(2)$
 $2 \times \text{factorial}(1)$
 $1 \times \text{factorial}(0)$

Result = 120

Why Recursion is Useful in Financial Forecasting

Financial forecasting often involves predicting future values based on current values and growth rates.

Formula:

$\text{Future Value} = \text{Present Value} \times (1 + \text{Growth Rate})$

Using recursion, we can calculate the value year after year.

2. Recursive Method for Financial Forecasting

Assume:

- Initial Investment = ₹10,000
 - Growth Rate = 10%
 - Forecast Period = 5 Years
-

Recursive Formula

$$FV(n) = FV(n-1) \times (1 + \text{Growth Rate})$$

Base Case

$$FV(0) = \text{Present Value}$$

3. Implementation

```
public class FinancialForecast {  
  
    public static double forecast(  
        double presentValue,  
        double growthRate,  
        int years) {  
  
        if(years == 0) {  
  
            return presentValue;  
        }  
  
        return forecast(  
            presentValue,  
            growthRate,  
            years - 1)  
            * (1 + growthRate);  
        }  
  
    public static void main(String[] args) {  
  
        double presentValue = 10000;  
  
        double growthRate = 0.10;  
  
        int years = 5;  
  
        double futureValue =  
            forecast(  
                presentValue,  
                growthRate,
```



```
        years);

    System.out.println(
        "Future Value = " +
        futureValue);
}
}
```

Output

Future Value = 16105.1

Step-by-Step Calculation

Year 0 = 10000

Year 1 = 10000×1.10
= 11000

Year 2 = 11000×1.10
= 12100

Year 3 = 12100×1.10
= 13310

Year 4 = 13310×1.10
= 14641

Year 5 = 14641×1.10
= 16105.1

Alternative Recursive Version

```
public class Forecast {  
  
    static double predict(  
        double value,  
        int years) {  
  
        if(years == 0)  
            return value;  
  
        return predict(  
            value * 1.08,  
            years - 1);  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println(  
            predict(  
                5000,  
                3));  
    }  
}
```

Time Complexity Analysis

Recursive Solution

forecast(value, rate, n)

The function executes once for each year.

$$T(n) = T(n-1) + O(1)$$

Therefore:

$$\text{Time Complexity} = O(n)$$

where **n** = **number of years**.

Space Complexity

Because each recursive call is stored in the call stack:

Space Complexity = $O(n)$

4. Optimization Techniques

Problem with Recursion

For very large values of **n**:

- Many recursive calls are generated.
 - Stack memory usage increases.
 - May cause `StackOverflowError`.
-

Optimization 1: Iterative Approach

```
public static double forecast(  
    double value,  
    double rate,  
    int years) {  
  
    for(int i = 1;  
        i <= years;  
        i++) {  
  
        value =  
            value * (1 + rate);  
    }  
  
    return value;  
}
```

Complexity

Time Complexity = $O(n)$

Space Complexity = $O(1)$

This is better than recursion.

Optimization 2: Dynamic Programming (Memoization)

Store previously calculated values.

```
double[] dp = new double[years + 1];
```

Avoids repeated calculations.

Optimization 3: Mathematical Formula

Compound Growth Formula:

Future Value =
Present Value $\times (1 + \text{Rate})^{\text{Years}}$

Java:

```
double futureValue =  
    presentValue *  
    Math.pow(  
        1 + rate,  
        years);
```

Complexity

Time Complexity = $O(1)$

Fastest approach.

Comparison of Approaches

Method	Time Complexity	Space Complexity
Recursion	$O(n)$	$O(n)$
Iteration	$O(n)$	$O(1)$
Dynamic Programming	$O(n)$	$O(n)$
Mathematical Formula	$O(1)$	$O(1)$

Conclusion

For Financial Forecasting:

- Recursion is easy to understand and implement.
- Iteration is more memory efficient.
- Mathematical formula using `Math.pow()` is the fastest and most practical solution for large-scale forecasting systems.